الهيئة الاتحادية للهوية والجنسية والجمارك وأمن المنافذ

FEDERAL AUTHORITY FOR IDENTITY, CITIZENSHIP, CUSTOMS & PORT SECURITY

ID Card Toolkit C/C++ Sample Build Instructions

Version 1.4

March 2026

CONFIDENTIAL

Federal Authority for Identity, Citizenship, Customs & Port Security (ICP)
United Arab Emirates

Table of Contents

Revision History

Version	Date	Description of Changes
1.0	17th May, 2018	Release Version 1.0.5
1.1	12th Jun, 2018	Release Version 1.0.6
1.2	15th Feb, 2019	Release Version 1.0.13
1.3	20th Dec, 2023	Release Version 3.0.0
1.4	Mar 2026	Toolkit v3.0.5. Added CMake build instructions. Updated Visual Studio instructions for VS 2019+. Updated Linux instructions.

Abbreviations

Abbreviation	Description
ICP	Federal Authority for Identity, Citizenship, Customs & Port Security
IDE	Integrated Development Environment
SDK	Software Development Kit
SP	Service Provider

1. Introduction

The Federal Authority for Identity, Citizenship, Customs & Port Security (ICP) developed the ID Card Toolkit to address the requirements of service providers (SP) to integrate, into their business applications: Identification, Authentication, Digital Signature and Non-repudiation services, around the capabilities of the Emirates ID Card. The Toolkit is comprised of a number of Software Development Kits (SDK) supporting different programming languages and platforms.

1.1. Purpose

This document is a developer's reference guide to create and build the sample C/C++ console application provided with the ICP ID Card Toolkit C/C++ SDK. Build instructions are provided for both Windows and Linux platforms.

2. Build Instructions for Windows (Visual Studio)

The following instructions are based on Visual Studio on Windows. The setup may vary slightly with different Visual Studio versions.

2.1. Prerequisites

- ICP ID Card Toolkit Windows C/C++ SDK
- IDE: Visual Studio 2026
- Windows 10 or later
- The following open-source libraries (pre-built or built from source):
  - OpenSSL (https://www.openssl.org/)
  - libxml2 (http://xmlsoft.org/)
  - xmlsec (https://www.aleksey.com/xmlsec/)
  - json-c (https://github.com/json-c/json-c)

2.2. Build Instructions

2.2.1. Step 1: Create a New Console Project

1. Open Visual Studio.
2. Go to **File > New > Project**.
3. Select **Console App** (or **Empty Project**) from the C++ project templates.
4. Set the project name and click **Create**.

2.2.2. Step 2: Place Sample Source Files

1. Right-click the project in Solution Explorer > **Open Folder in File Explorer**.
2. Copy all sample source files from the Toolkit distribution into the project folder.
3. In Visual Studio, right-click the project > **Add > Existing Item** and select all copied source files.

After adding, the project should contain:

```
EIDAToolkitConsole/
  EIDAToolkitConsole.c
  EIDAToolkitConsole.h
  EIDAToolkitHandler.c
  EIDAToolkitOptions.c
  EIDAToolkitOptions.h
  LibXmlParser.c
  LibXmlParser.h
```

2.2.3. Step 3: Configure Include Paths

1. Right-click the project > **Properties**.
2. Under **Configuration Properties > C/C++ > General**, add the following to **Additional Include Directories**:
   - Toolkit `include` directory (from the SDK distribution)
   - OpenSSL include directory
   - libxml2 include directory

- xmlsec include directory
- json-c include directory

2.2.4. Step 4: Configure Linker Settings

1. Under **Configuration Properties > Linker > General**, add the following to **Additional Library Directories**:
   - Path to `EIDAToolkit.lib` (from the `libs` folder of the distribution)
   - Paths to the open-source library directories
2. Under **Configuration Properties > Linker > Input**, add the following to **Additional Dependencies**:
   - `EIDAToolkit.lib`
   - OpenSSL libraries
   - json-c library
   - libxml2 library
   - xmlsec library
   - `crypt32.lib` (Windows system library, required by OpenSSL)

2.2.5. Step 5: Build

1. Select the target platform (**x86** or **x64**) and configuration (**Debug** or **Release**).
2. Build the project (**Build > Build Solution** or `Ctrl+Shift+B`).

> **Note:** The Toolkit SDK library is protected against debugging. Toolkit API calls cannot be stepped through, but your application code can be debugged normally.

2.2.6. Step 6: Set Up Runtime Dependencies

Place the following files in the same directory as the built executable:

1. **Toolkit library:** `EIDAToolkit.dll` from the `libs` folder (matching build architecture).
2. **Plugin libraries:** Required plugin DLLs from the `plugins` folder (matching build architecture).
3. **Configuration file:** `config_ap` in the executable directory.
4. **SP configuration:** Place SP-specific configuration files in the subdirectory specified by the `environment` parameter in `config_ap`.

> **Note:** If `config_ap` is not provided, default configuration is used and all SP configuration files must be placed directly in the executable directory.

2.2.7. Step 7: Run

Run the sample from Visual Studio or execute the built `.exe` directly from the output directory.

3. Build Instructions for Linux

The following instructions are for building on Linux. The sample can be built using any C compiler and IDE, or from the command line.

3.1. Prerequisites

- ICP ID Card Toolkit Linux C/C++ SDK
- GCC compiler
- IDE (optional): any C/C++ IDE (e.g., VS Code, CLion, NetBeans)
- Ubuntu or other Linux distribution
- The following open-source libraries (install via package manager or build from source):
  - OpenSSL
  - libxml2
  - xmlsec
  - json-c

On Ubuntu/Debian, install development packages:

```
sudo apt-get install libssl-dev libxml2-dev libxmlsec1-dev libjson-c-dev
```

3.2. Build Instructions

3.2.1. Step 1: Set Up Project Directory

Create a project directory and copy the sample source files from the Toolkit distribution:

```
mkdir EIDAToolkitConsole && cd EIDAToolkitConsole
cp /path/to/toolkit/samples/*.c .
cp /path/to/toolkit/samples/*.h .
```

3.2.2. Step 2: Build with GCC

Compile the sample with the Toolkit library and required dependencies:

```
gcc -o EIDAToolkitConsole \
    EIDAToolkitConsole.c \
    EIDAToolkitHandler.c \
    EIDAToolkitOptions.c \
    LibXmlParser.c \
    -I/path/to/toolkit/include \
    -L/path/to/toolkit/libs \
    -lEIDAToolkit \
    -lssl -lcrypto \
    -lxml2 -lxmlsec1 \
    -ljson-c \
    -lpthread -lm -ldl
```

> **Note:** Adjust the include and library paths to match your Toolkit installation and open-source library locations.

3.2.3. Step 3: Set Up Runtime Dependencies

1. Copy `libEIDAToolkit.so` and plugin shared libraries to the executable directory or a directory in `LD_LIBRARY_PATH`.
2. Place `config_ap` and SP-specific configuration files in the executable directory.

```
export LD_LIBRARY_PATH=/path/to/toolkit/libs:$LD_LIBRARY_PATH
```

3.2.4. Step 4: Run

```
./EIDAToolkitConsole
```

4. Build Instructions using CMake (Cross-Platform)

The Toolkit v3.0.5 distribution includes CMake build support for the sample application, providing a cross-platform build that works on Windows, Linux, and macOS.

4.1. Prerequisites

- CMake 3.20 or above
- A C compiler (MSVC on Windows, GCC/Clang on Linux/macOS)
- The same open-source library dependencies as listed above

4.2. Build Instructions

4.2.1. Step 1: Configure

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
```

On Windows with Visual Studio:

```
cmake -B build -G "Visual Studio 18 2026" -A x64
```

4.2.2. Step 2: Build

```
cmake --build build --config Release
```

4.2.3. Step 3: Run

The built executable and required runtime dependencies are placed in the build output directory. Set up the configuration files as described in the platform-specific sections above, then run the executable.